

AWS CLOUD ENGINEERING

Capstone Project

Deploying a Production-Grade 3-Tier Microservice
Infrastructure on Amazon Web Services

Student Name: Felicia Omonike Owolabi

Date: 29th of July 2026

AWS Account ID (last 4 digits): 1738

Provisioning Method: AWS Management Console

Section 1 — Architecture Diagram

Insert your annotated architecture diagram below, labeled with your actual CIDR blocks and resource names.

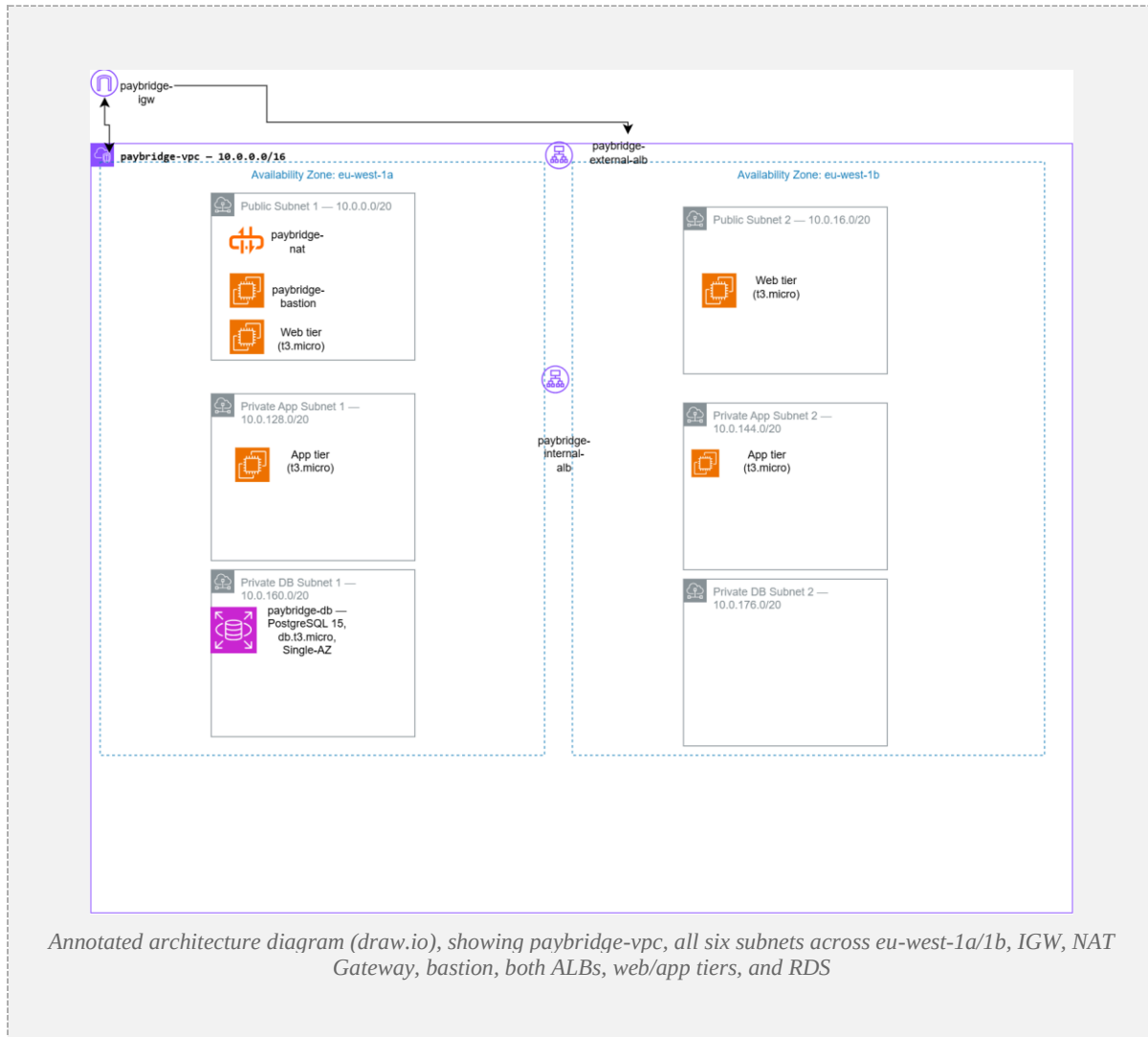


Figure 1: PayBridge VPC architecture — 10.0.0.0/16, eu-west-1, Multi-AZ

Section 2 — Task 1: VPC & Networking Foundation

This section documents the networking layer of the PayBridge infrastructure: the VPC, six subnets across two Availability Zones, Internet Gateway, NAT Gateway, and the two route tables that separate public and private traffic flows.

Q1: Why does the database tier need a separate private subnet rather than sharing the application tier subnet?

Separating the DB tier into its own subnets enforces network-level isolation on top of Security Group isolation, giving the architecture defense in depth. Even if the appserver-sg were ever misconfigured to allow broader access, a separate subnet allows a distinct Network ACL to be applied as an additional layer of control, independent of Security Group rules.

It also mirrors real production practice: DB subnets typically have stricter routing requirements and are often subject to different compliance and audit boundaries — for example, PCI-DSS scope in a fintech context like PayBridge. Finally, it makes IP address planning and access review much clearer: anyone auditing the VPC can immediately see which CIDR ranges hold sensitive data stores versus application logic, without inspecting every instance's Security Group individually.

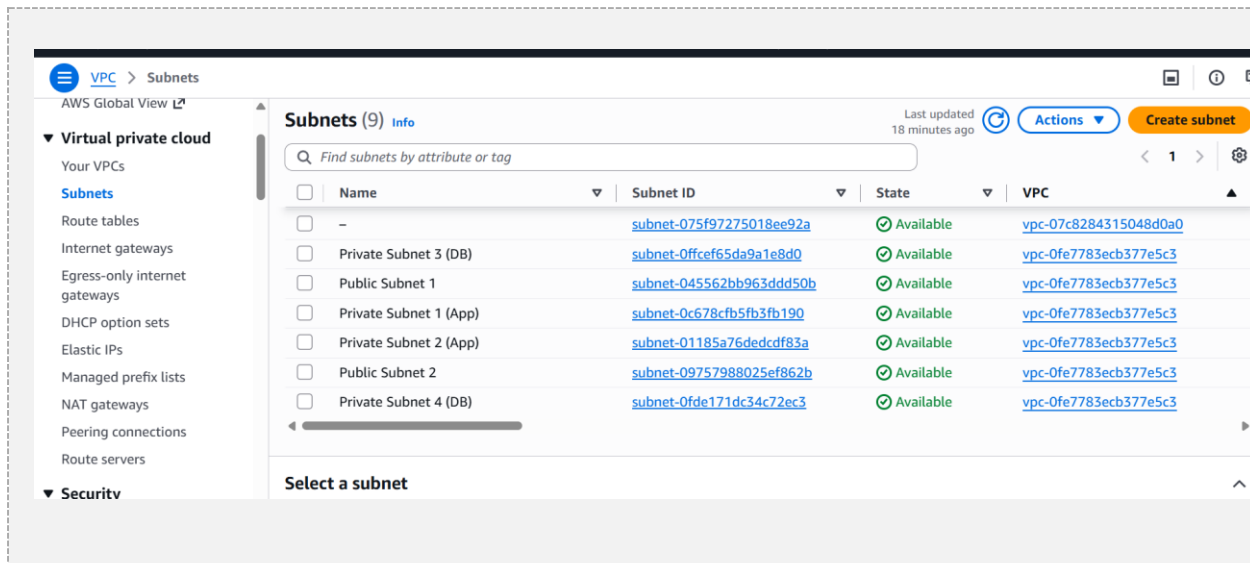


Figure 1.1: Subnets list filtered to paybridge-vpc, showing all 6 subnets split across Public, Private (App), and Private (DB) tiers and two Availability Zones

Q2: How many route tables does your design need at minimum, and why?

Two: public-rtb and private-rtb. The public subnets need a route to the Internet Gateway (0.0.0.0/0 → IGW) so the ALB and web-tier EC2 instances can be reached from, and reach, the internet directly. The private subnets need a different default route — to the NAT Gateway rather than the IGW — so app and DB tier resources can initiate outbound connections, such as package updates or calls to AWS APIs, without being directly reachable from the internet.

Since a subnet can only be associated with one route table at a time, and public and private subnets require genuinely different default routes, two is the functional minimum for this architecture. A design could use more route tables for finer-grained control — for example, one per AZ or per tier — but two satisfies the core public/private routing requirement.

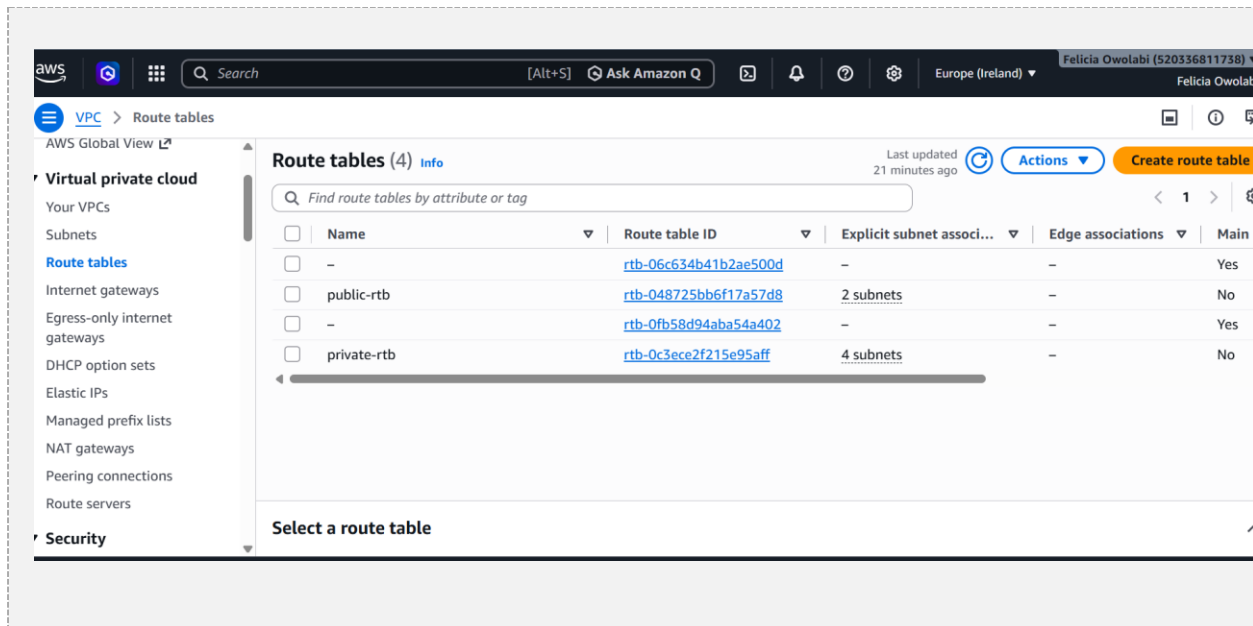


Figure 1.2: Route tables showing public-rtb (2 subnet associations) and private-rtb (4 subnet associations)

Q3: What would happen if the NAT Gateway was placed in a private subnet instead of a public subnet?

It would fail to provide internet connectivity at all. A NAT Gateway itself needs a route to the Internet Gateway in order to translate and forward outbound traffic from private instances to the internet. If it were placed in a private subnet, that subnet's route table would point 0.0.0.0/0 to another NAT Gateway or nowhere, not to an IGW — so the NAT Gateway would have no path out to the internet itself, defeating its purpose.

It would also fail during creation or validation in some cases, since AWS requires an Elastic IP for a NAT Gateway, which only makes sense for a publicly routable subnet. In short, NAT Gateways must live in public subnets because they act as the 'middleman' translating private traffic to public traffic, and they need direct IGW access to do that job.

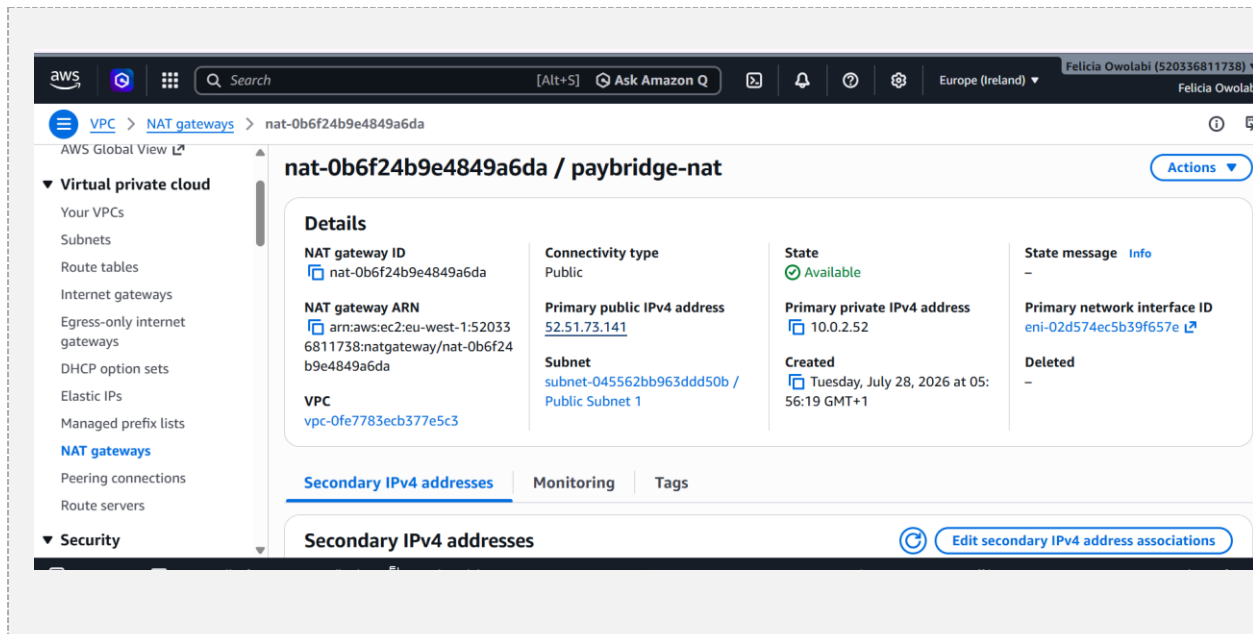


Figure 1.3: NAT Gateway (paybridge-nat) detail page, confirming it is located in Public Subnet 1

Q4: Your subnets use /20 masks. How many usable host IPs does each subnet provide? Show your calculation.

A /20 CIDR block has $32 - 20 = 12$ host bits, giving $2^{12} = 4,096$ total IP addresses per subnet.

AWS reserves 5 IP addresses in every subnet for internal networking purposes: the network address (e.g. 10.0.0.0), the VPC router (e.g. 10.0.0.1), the DNS server (e.g. 10.0.0.2), a reservation for future use (e.g. 10.0.0.3), and the broadcast address (e.g. 10.0.15.255).

So usable IPs = $4,096 - 5 = 4,091$ usable host IPs per subnet. This matches the AWS Console's own display when each subnet was created, which showed '4,096 IPs' next to the CIDR field — that figure is the total count, not the usable count, which is a common point of confusion worth clarifying explicitly in this report.

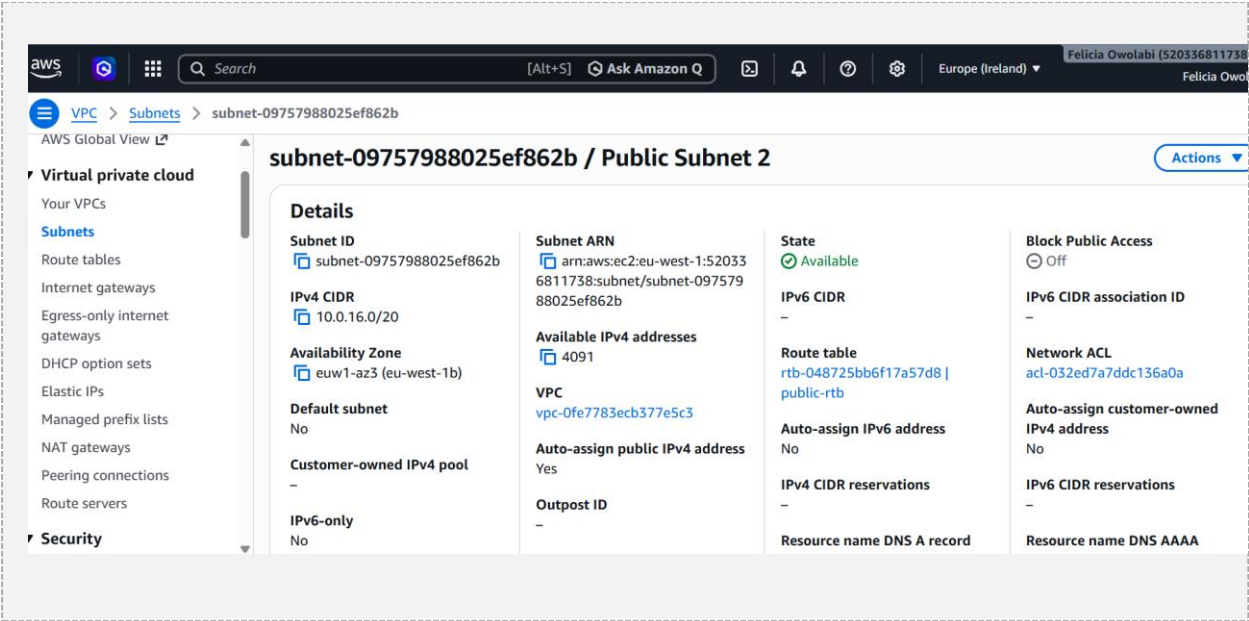


Figure 1.4Public Subnet 2 detail page showing 4,091 available IPv4 addresses on a /20 CIDR block

Section 3 — Task 2: Security Groups

This section documents the four Security Groups built for the PayBridge infrastructure — bastion-sg, webserver-sg, appserver-sg, and database-sg — each scoped to the principle of least privilege, with rules referencing other Security Groups rather than open IP ranges wherever possible.

Q1: Why is it dangerous to allow 0.0.0.0/0 as the SSH inbound source on webserver-sg?

Allowing SSH from 0.0.0.0/0 exposes port 22 to every IP address on the internet, turning web-tier instances into a constant target for automated brute-force login attempts and credential-stuffing bots — this happens within minutes of a public IP going live, not hypothetically.

Even with strong key-based authentication rather than passwords, an open SSH port is unnecessary attack surface: it gives attackers a foothold to attempt exploits against the SSH daemon itself, and it means every login attempt, successful or not, has to be filtered out of logs from scanner noise. Restricting SSH to a single bastion host, as this architecture does, means there is exactly one narrow, monitored entry point into the private network, rather than every public instance being independently exposed.

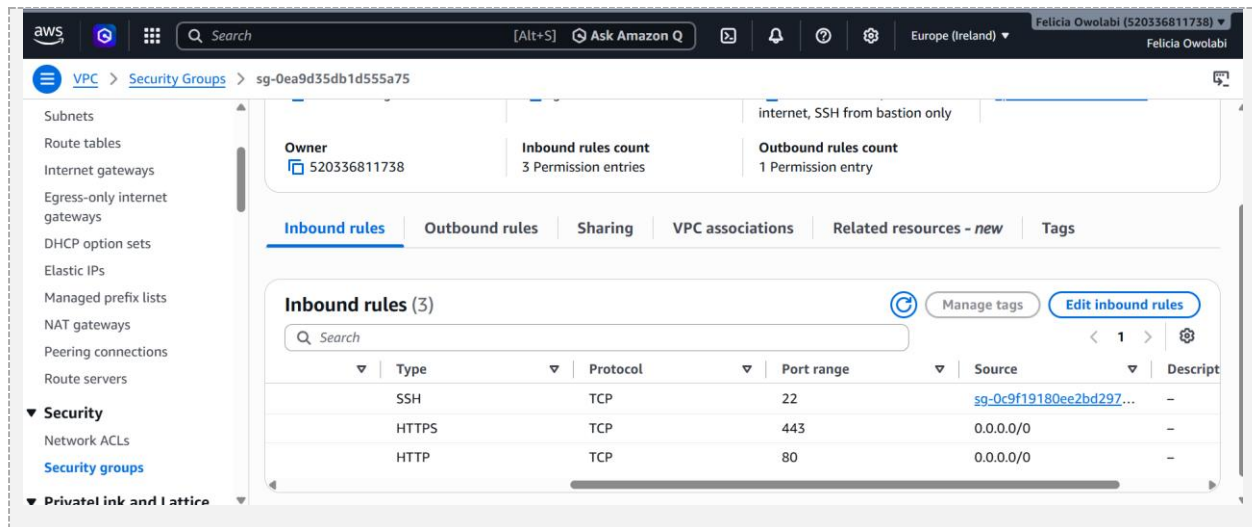


Figure 2.1: webserver-sg inbound rules — SSH restricted to bastion-sg as the source, not 0.0.0.0/0

Q2: What is the role of the Bastion Host, and why is it placed in a public subnet?

The bastion host acts as a single, controlled entry point for administrative SSH access into the private network. Instead of exposing every EC2 instance's SSH port to the internet, only the bastion is reachable from outside, and even then only from a specific known IP. Once authenticated to the bastion, an operator can then SSH onward from the bastion into the private app-tier or web-tier instances, using the bastion as a 'jump box.'

It must live in a public subnet because it needs a public IP and a route to the Internet Gateway to be reachable by an admin working remotely — without that, there would be no way to reach it at all.

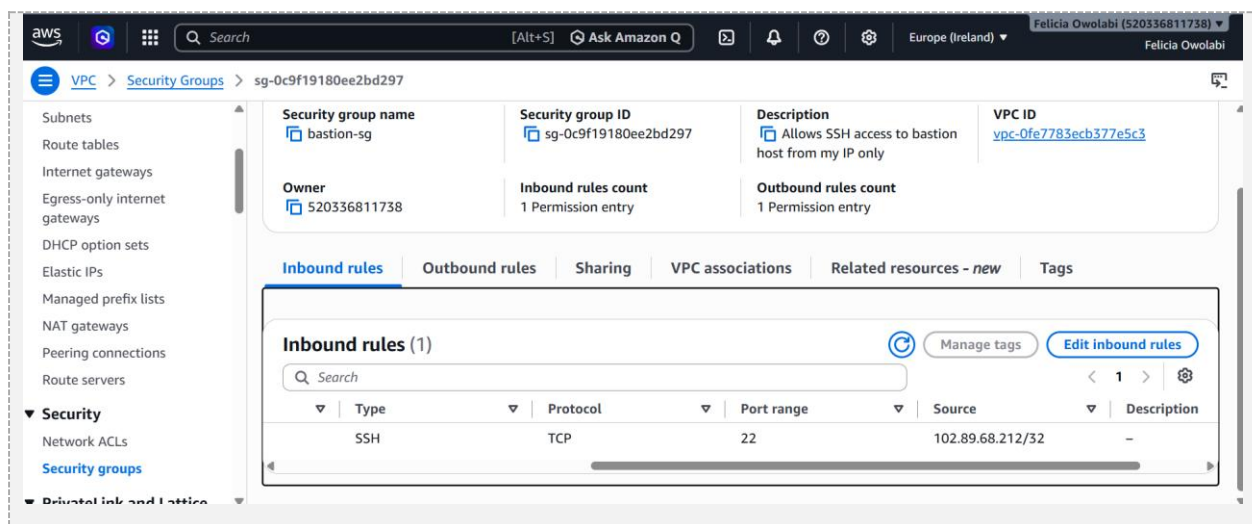


Figure 2.2 bastion-sg inbound rules — SSH restricted to a single /32 IP address (my own)

Q3: Could you replace the Bastion Host with AWS Systems Manager Session Manager? What are the trade-offs?

Yes — Session Manager allows getting a shell into an EC2 instance through the AWS Console or CLI using IAM permissions, without opening any inbound SSH port at all, since the instance connects outbound to the Systems Manager service rather than accepting inbound connections.

Trade-offs: Session Manager requires the SSM Agent running on each instance and an IAM role attached with the right permissions, plus, for private instances, either a NAT Gateway or a VPC endpoint for Systems Manager so the agent can reach the SSM service. In exchange, SSH key management is eliminated entirely, every session is automatically logged to CloudWatch or S3 for audit purposes, and port 22 is removed from the attack surface completely — arguably a stronger security posture than a bastion host, though it adds a dependency on correct IAM configuration and represents a shift in tooling that traditional sysadmins may need to adjust to.

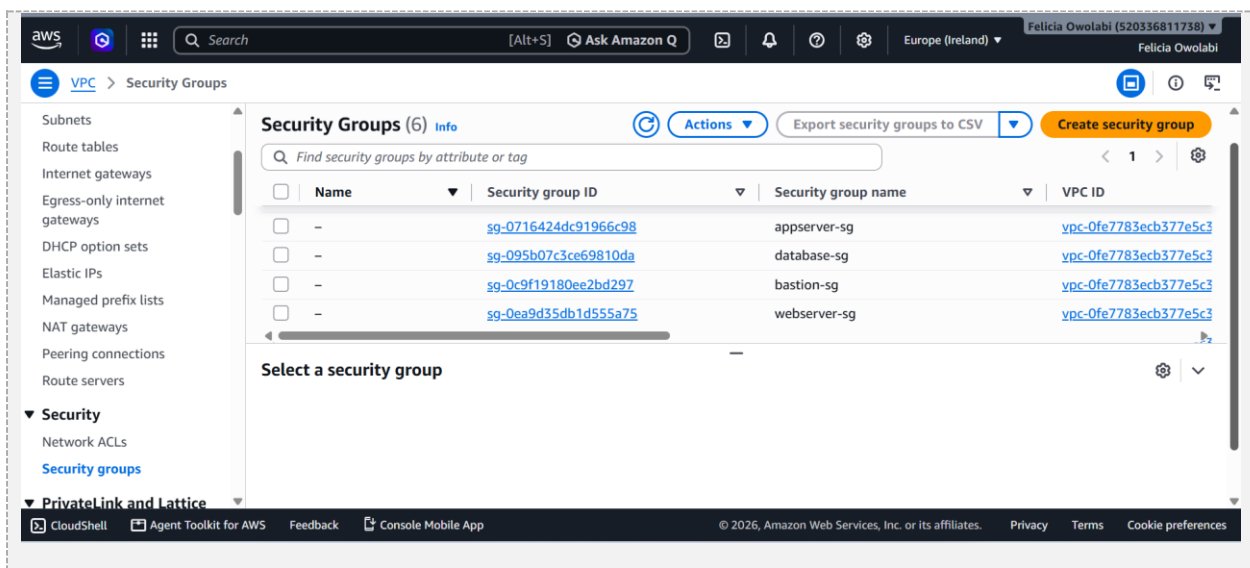


Figure 2.3: Security Groups list — all four groups built for the bastion-based access approach used in this project

Q4: The database-sg has no inbound rule for port 22. Why? Would it ever need one?

RDS is a managed service — AWS operates the underlying database instance's OS, patching, and infrastructure, so there is no SSH access to an RDS instance at all; it is simply not exposed as an option, and it is not needed for administration since connections are made via the PostgreSQL protocol on port 5432 using database credentials, not by logging into a shell.

If this were a self-managed EC2-based database instead of RDS, SSH access would be needed for OS-level maintenance, and port 22 would be added scoped to bastion-sg only, following the exact same least-privilege pattern used for the app and web tiers. But since the DB tier here is a managed RDS instance, port 22 is correctly absent.

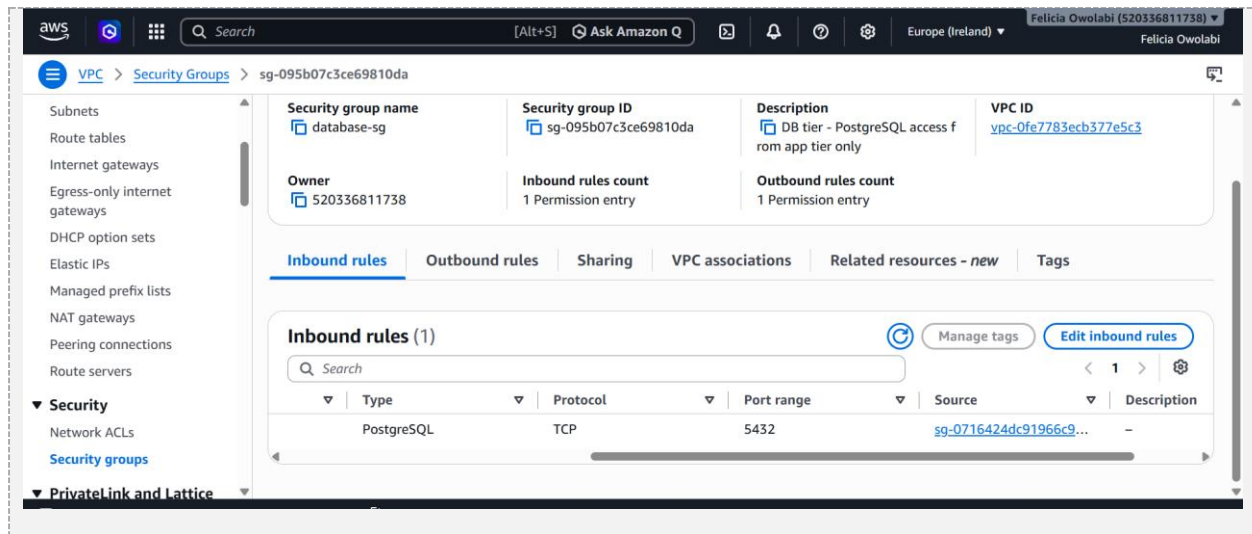


Figure 2.4: database-sg inbound rules — only PostgreSQL (5432) permitted, sourced from appserver-sg only

Section 4 — Task 3: Compute (EC2, Launch Templates, ASGs)

This section documents the compute layer of the PayBridge infrastructure: the Bastion Host, the Web-Tier and App-Tier Launch Templates, and the two Auto Scaling Groups (paybridge-web-asg and paybridge-app-asg) that keep each tier running at the desired capacity across both Availability Zones.

Q1: What is the difference between a Launch Template and a Launch Configuration? Why does AWS recommend Launch Templates?

A Launch Configuration is an older, immutable resource: once created it cannot be edited or versioned, so any change to instance type, AMI, or user data requires creating an entirely new configuration and re-pointing the ASG to it. A Launch Template is the modern replacement — it supports versioning (multiple numbered versions of the same template, with a “default” and “latest” pointer), lets you mix instance types for Spot/On-Demand diversification, and supports newer EC2 features like multiple network interfaces, dedicated hosts, and placement groups that Launch Configurations never received updates for.

AWS recommends Launch Templates because they are actively developed and because versioning makes iteration safe: a new version of paybridge-web-It could be created with an updated AMI, tested, and rolled back instantly if something breaks, without needing to delete and recreate the entire resource. Launch Configurations are effectively in maintenance mode and AWS no longer adds new EC2 capabilities to them.

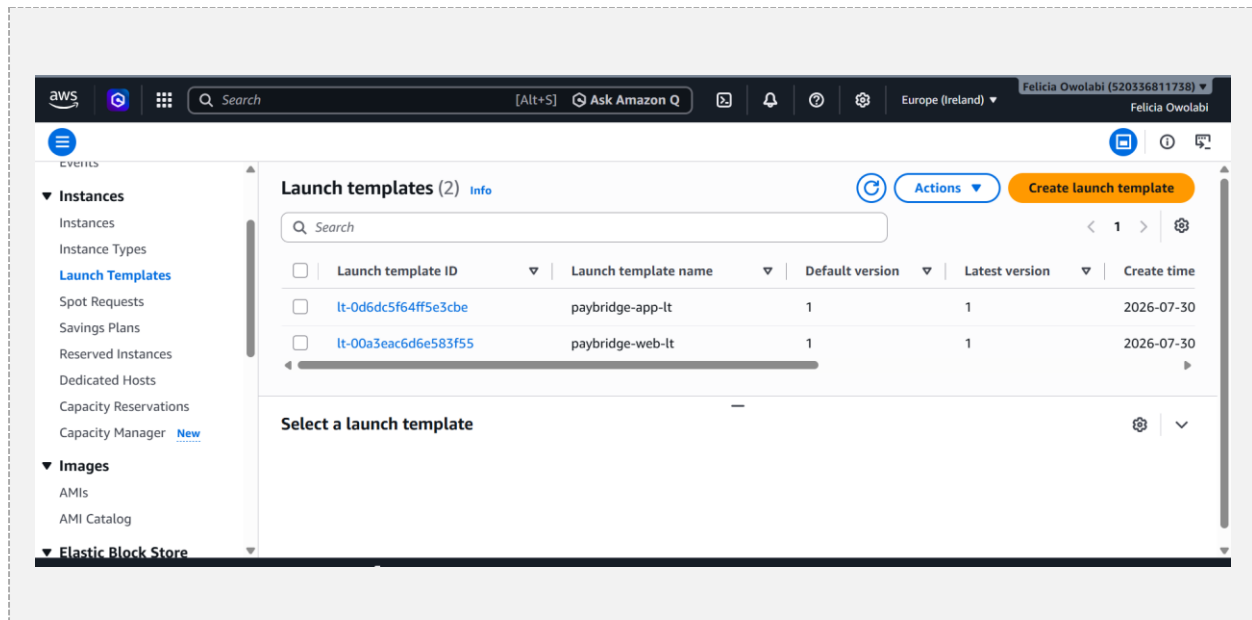


Figure 3.1: paybridge-web-lt and paybridge-app-lt launch templates list, showing both created

Q2: Your ASG has Desired: 2, Min: 2, Max: 4. Describe the exact event that would cause it to scale to 3 or 4 instances.

With no scaling policy attached (as configured in this project), the ASG will only add instances beyond the desired count in one scenario: if a target-tracking or step-scaling policy is later added — for example, “keep average CPU at 50%” — and the actual average CPU utilization across the group’s instances rises above that threshold for the configured evaluation period, CloudWatch triggers an alarm that the ASG responds to by launching additional instances, up to the Max of 4.

Without a scaling policy, the only way the count changes is manual: an operator raising the Desired capacity value directly in the console or CLI, or an instance failing a health check and being replaced (which keeps the count at 2, it does not grow it). For true elasticity under load — the 10,000-concurrent-user scenario described in the brief — a CPU- or request-count-based target tracking policy would need to be added as a next step.

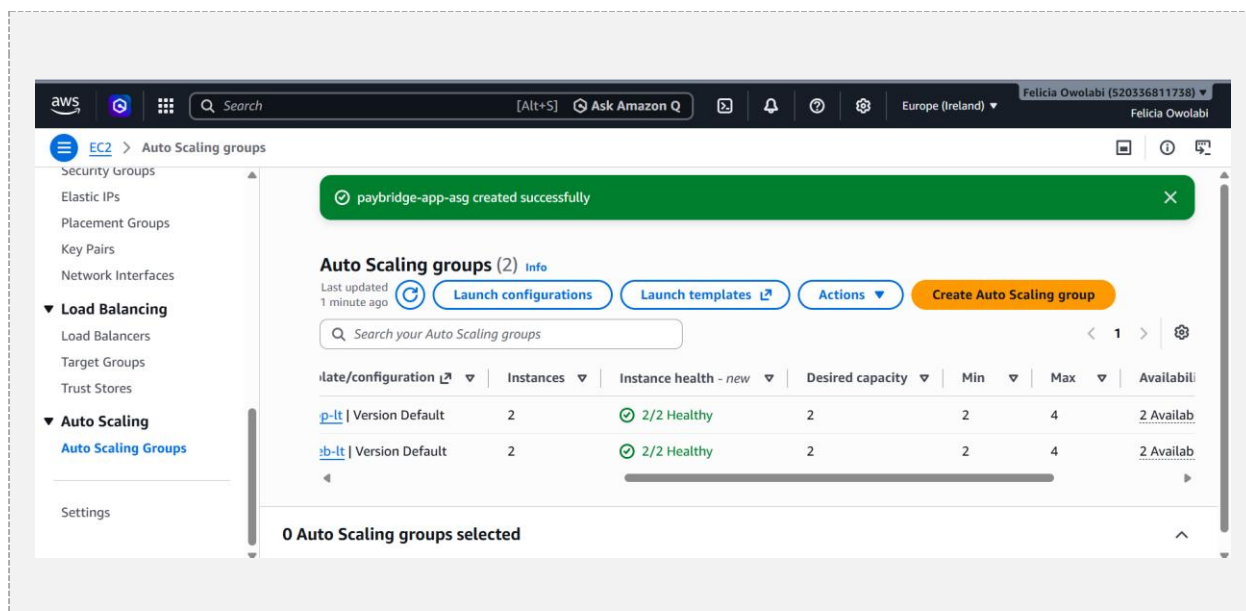


Figure 3.2: paybridge-web-asg and paybridge-app-asg, Desired/Min/Max set to 2/2/4

Q3: Why do the app-tier EC2 instances have no public IP? How do they still manage to download packages from the internet?

They have no public IP because they sit in private subnets, by design — the whole point of the 3-tier architecture is that the application layer should never be directly reachable from the internet, only from the web tier via the internal ALB. Removing the public IP entirely, rather than just relying on Security Group rules, means there is no routable path to these instances from outside the VPC at all, even if a Security Group were ever misconfigured.

They still reach the internet outbound (for yum update and pip3 install flask in the user data script) because the private route table (private-rtb) routes 0.0.0.0/0 to the NAT Gateway sitting in the public subnet. The NAT Gateway translates the private instance's outbound request to its own public Elastic IP, sends it out via the Internet Gateway, and routes the response back — all without ever exposing an inbound path to the app-tier instance itself.

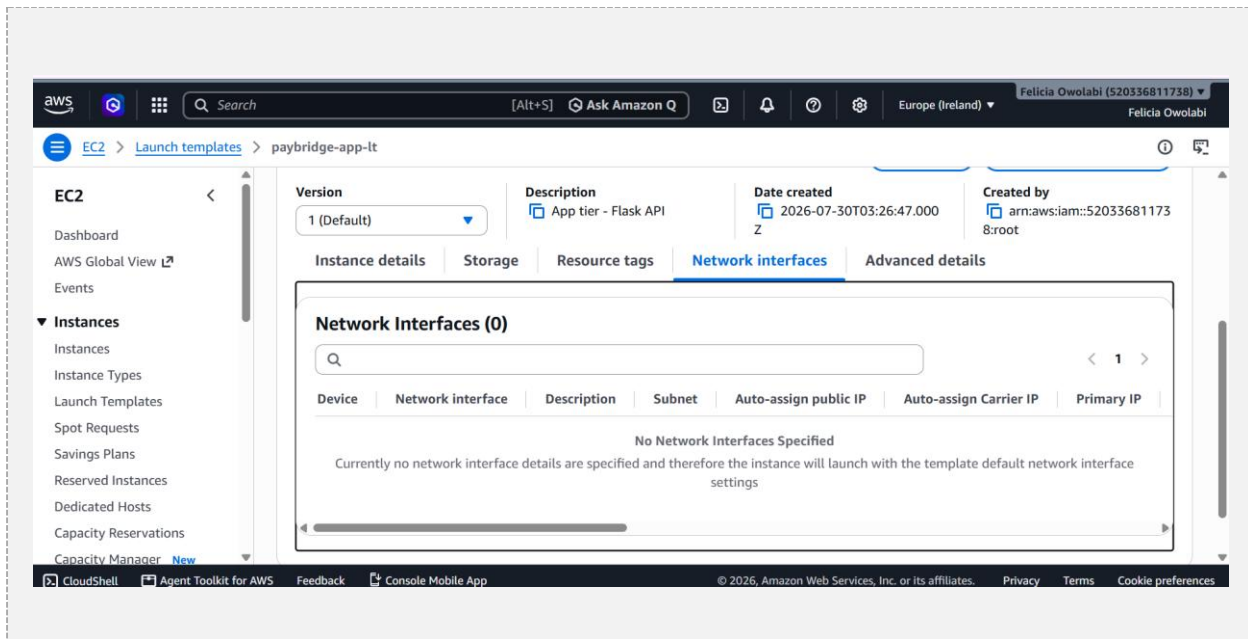
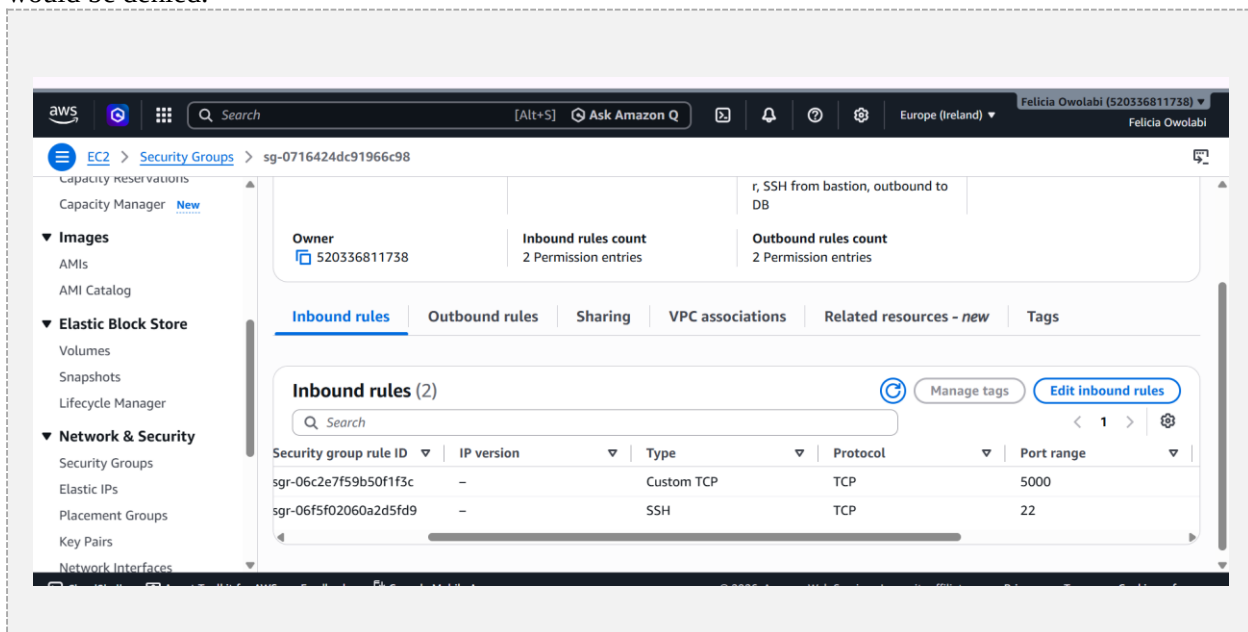


Figure 3.3 paybridge-app-1t network interfaces tab — no network interface specified, so instances inherit the private subnet's default (no public IP)

Q4: If you SSH into a web-tier EC2 directly from the bastion, which security group rule allows this? Write out the exact rule.

The rule lives on webserver-sg, in the inbound rules: Type: SSH, Protocol: TCP, Port: 22, Source: bastion-sg. Because the source is specified as a Security Group reference rather than a CIDR block, this rule only permits traffic from instances that have bastion-sg attached — in this case, exactly one instance, paybridge-bastion. Any other instance, even one with the exact same private IP range but a different Security Group, would be denied.



- *Figure 3.4: webserver-sg inbound rules, showing the SSH rule sourced from bastion-sg*

Section 5 — Task 4: Application Load Balancers

This section documents the load balancing layer of the PayBridge infrastructure: the external, internet-facing ALB (paybridge-external-alb) that fronts the web tier, and the internal ALB (paybridge-internal-alb) that routes traffic from the web tier to the app tier — keeping the application layer unreachable from outside the VPC while still allowing it to be load-balanced across both Availability Zones.

Q1: What is the difference between an internet-facing and an internal load balancer in terms of IP assignment?

An internet-facing ALB is assigned public IP addresses in addition to private ones — its DNS name resolves to public IPs, and it lives in the public subnets so it's reachable directly from the internet. An internal ALB only ever receives private IP addresses from the subnets it's deployed in; its DNS name resolves to private IPs only, so it's reachable exclusively from within the VPC (or a peered/connected network).

This is exactly why paybridge-internal-alb can route to the app tier without ever exposing it externally — it has no public-facing address at all, so there is no internet path to it even if someone had its DNS name.

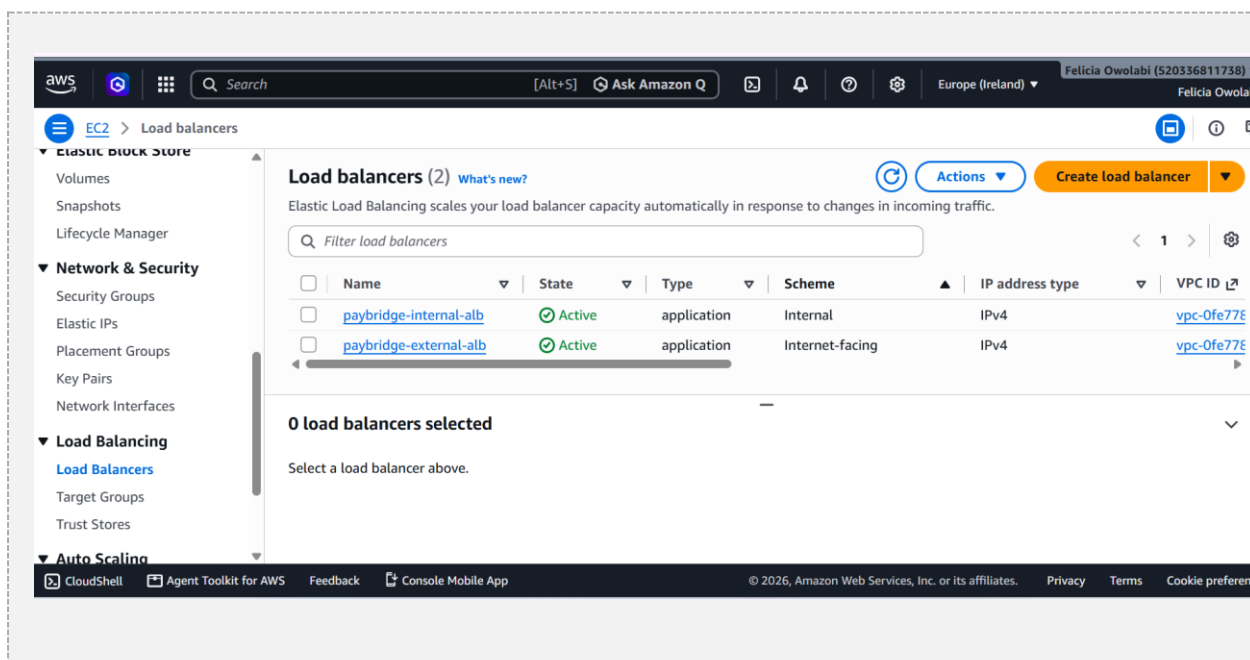


Figure 4.1: Load Balancers list — external ALB (internet-facing) and internal ALB (internal), side by side

Q2: Your ALB performs health checks every 30 seconds. An instance fails 3 consecutive checks. What happens to traffic for that instance?

The ALB marks that target as unhealthy in the target group and immediately stops routing new requests to it — the remaining healthy targets absorb the traffic. The instance stays registered rather than being deregistered, so if it recovers and passes the configured number of consecutive successful checks, the ALB automatically resumes sending it traffic.

This is separate from ASG health checks — if the ASG is also configured to use ELB health checks rather than just EC2 status checks, a target that stays unhealthy long enough can additionally get terminated and replaced by the ASG, independent of the ALB's own retry behaviour.

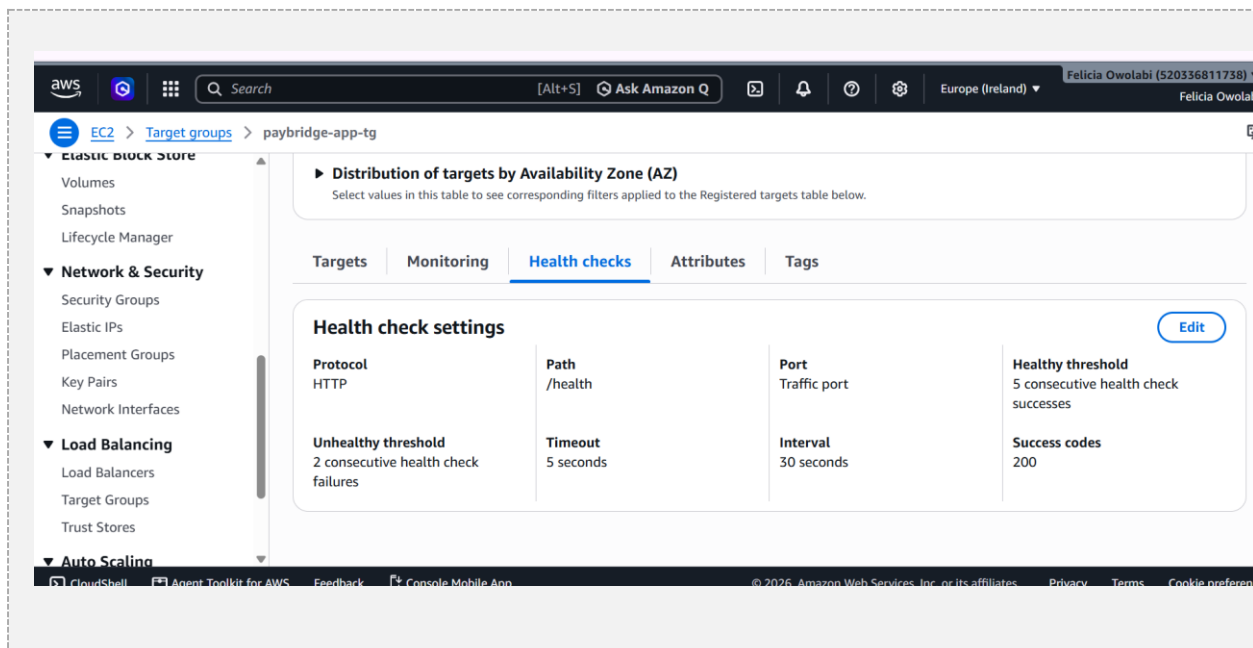


Figure 4.2: Target group health check configuration — unhealthy threshold count

Q3: Could you replace both ALBs with a single ALB using path-based routing (e.g. /api → app tier, / → web tier)? What are the security implications?

Technically yes — a single internet-facing ALB with listener rules (/api/* → app-tg, / → web-tg) would work at the routing level. But it would mean the app tier becomes reachable from the public internet through that ALB, even though the app-tier EC2 instances themselves would still sit in private subnets — the security boundary shifts from "the app tier has no internet-facing entry point at all" to "the app tier is reachable, just via a shared load balancer."

That's a meaningfully weaker posture for a fintech application: it collapses the tier separation the whole architecture is built around, removes the ability to apply different WAF rules or network ACLs per tier's ALB, and increases blast radius if the shared ALB is ever misconfigured.

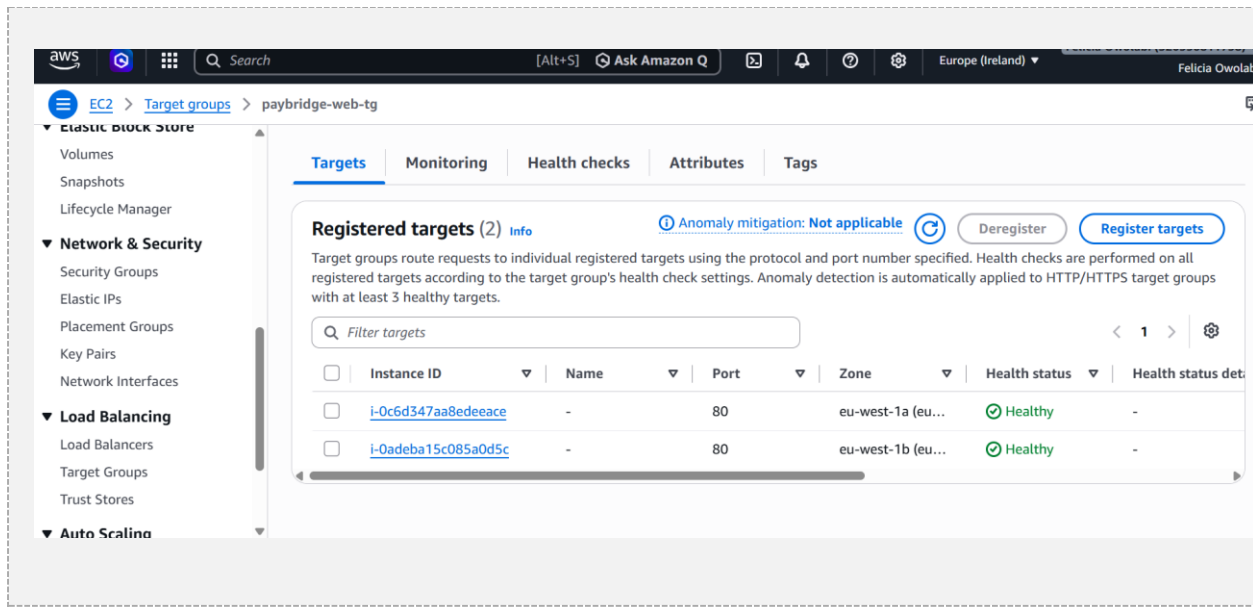


Figure 4.3: paybridge-web-tg registered targets — 2 healthy instances on port 80, confirming the web tier's separate target group

Q4: The internal ALB has no internet route. If an app-tier instance restarts, can the ALB still reach it? Explain why.

Yes. ALB-to-target communication happens entirely over the VPC's private network using private IP addresses — it never needs a route to the Internet Gateway or NAT Gateway to reach targets within its own VPC.

When the app-tier instance restarts, or the ASG launches a replacement, it re-registers with the target group using its private IP, and as soon as it passes health checks the internal ALB resumes routing to it — completely independent of any internet route.

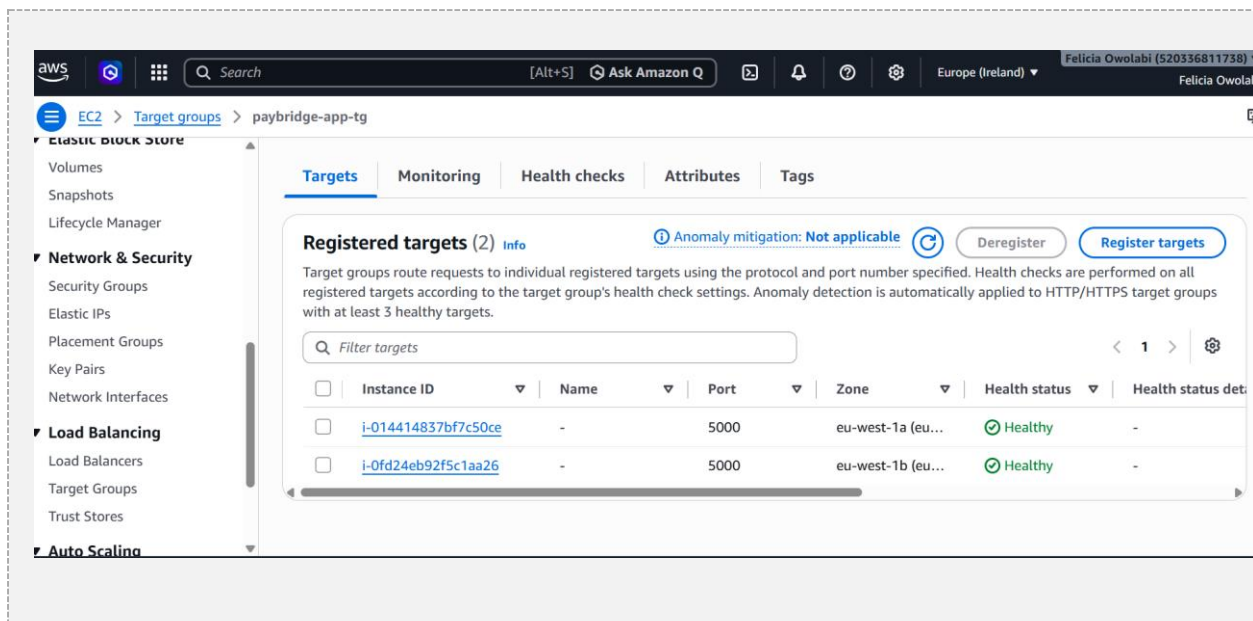


Figure 4.4: paybridge-app-tg registered targets — 2 healthy instances on port 5000, confirming the internal ALB reaches the app tier

Section 6 — Task 5: RDS PostgreSQL Database

This section documents the database layer of the PayBridge infrastructure: the DB Subnet Group (paybridge-db-sg-group), the Single-AZ RDS PostgreSQL instance (paybridge-db), and a verified end-to-end connection test from an app-tier EC2 instance to the database, proving the full path from application layer to data layer works as designed.

Build note: Multi-AZ and backup retention constraints

This AWS account is on a Free Plan, which enforces two account-level limits that affected this task: (1) Multi-AZ deployment is disabled account-wide — the Multi-AZ option remained greyed out regardless of DB template (Free tier, Dev/Test, and Production were all tested), and (2) the backup retention period is capped below the project's requested 7 days, requiring it to be reduced to 1 day before the database would create successfully. Both are genuine account-level constraints rather than design choices, and are addressed directly in the reflection answers below, particularly Q1 on RPO/RTO impact.

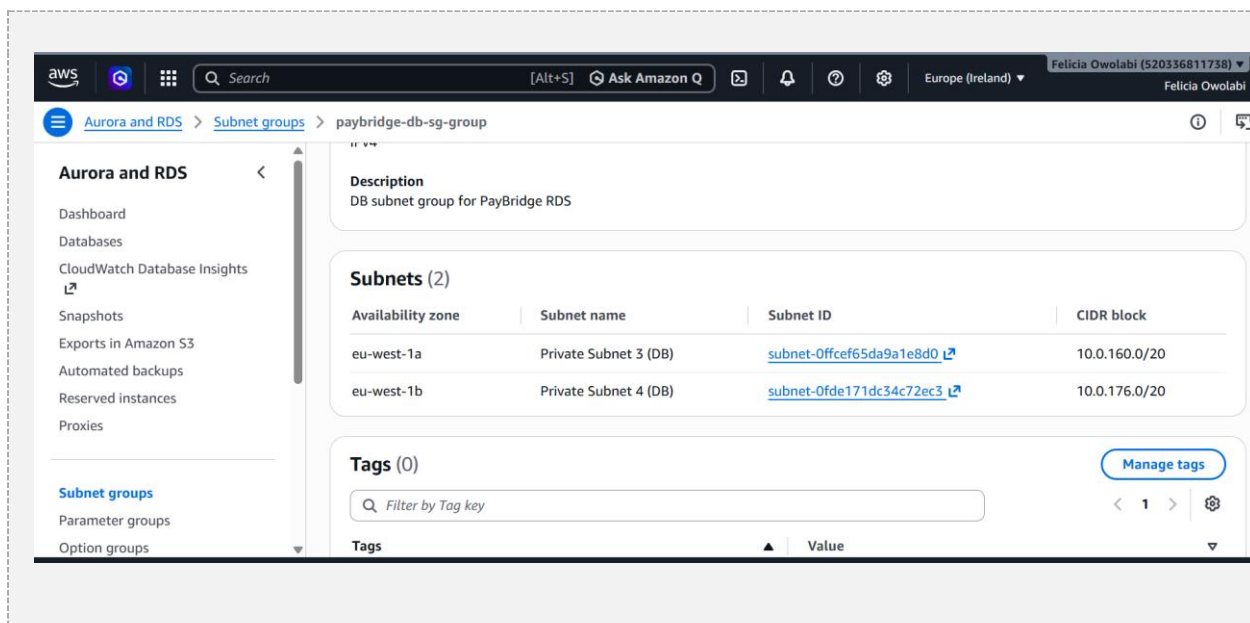


Figure 5.1: paybridge-db-sg-group subnet group detail, confirming Private Subnet 3 (DB) and Private Subnet 4 (DB) across eu-west-1a and eu-west-1b

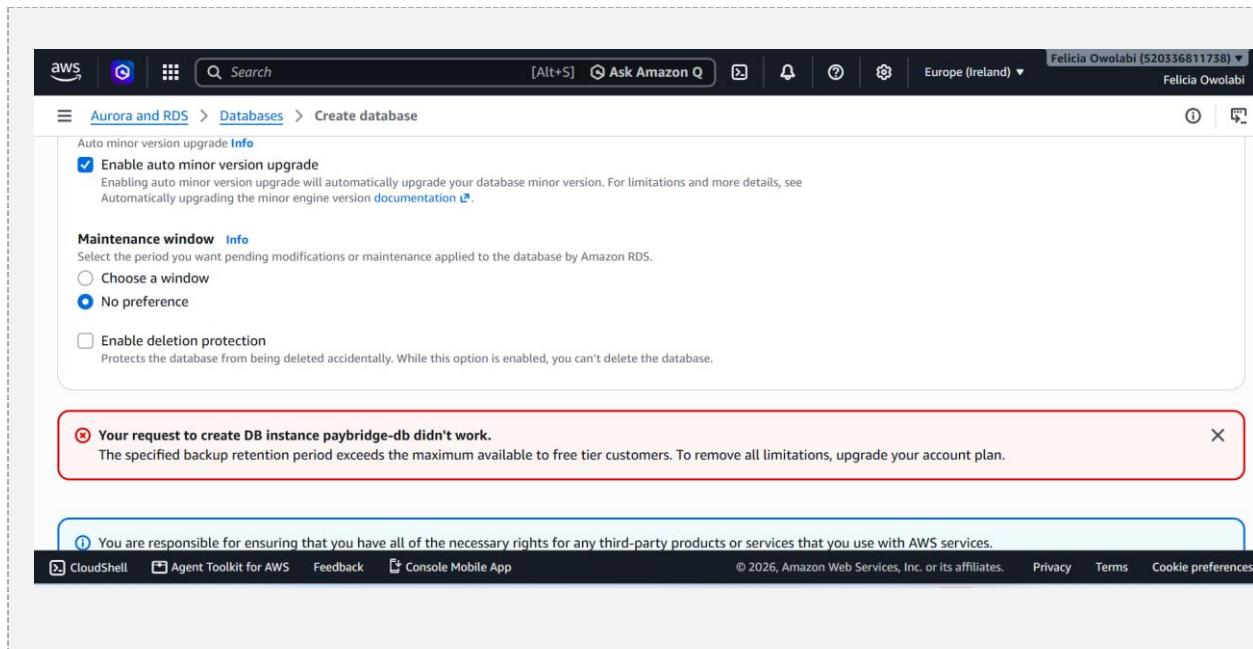


Figure 5.2: RDS 'Create database' error: backup retention period exceeding free-tier maximum, before correction to 1 day

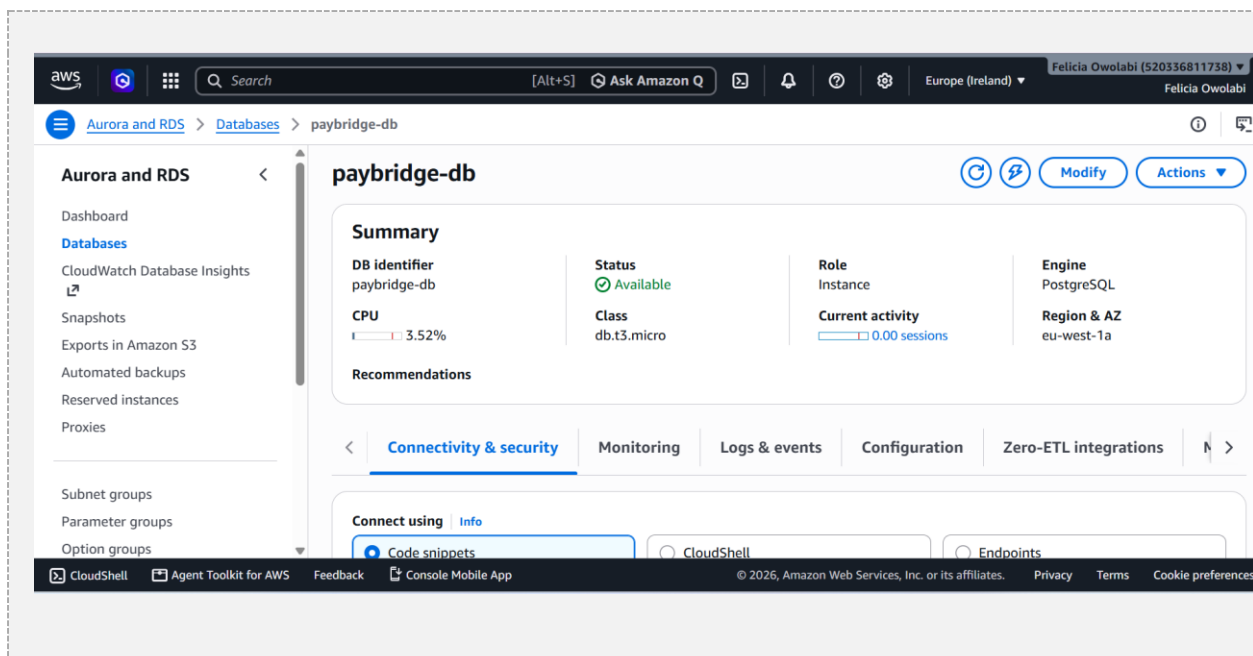


Figure 5.3: paybridge-db Summary panel showing Status: Available, Class: db.t3.micro, Engine: PostgreSQL, Region & AZ: eu-west-1a

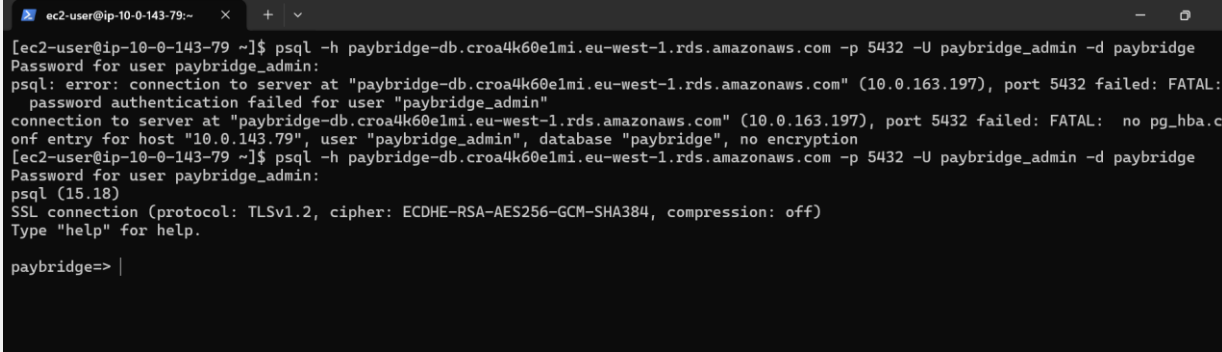
Q1: What is RPO (Recovery Point Objective) and RTO (Recovery Time Objective)? How does Multi-AZ RDS affect each?

RPO is how much data you can afford to lose, measured in time. RTO is how long you can afford to be down. Multi-AZ keeps a live standby in a second AZ, so failover is automatic and RPO is near zero. My database is Single-AZ because the free-plan account blocks Multi-AZ, so my RPO depends on the daily

backup instead of real-time replication, and recovery means restoring a snapshot rather than an instant failover.

Q2: Your database is in a private subnet with no public accessibility. How will the app-tier EC2 connect to it? Write the connection string format.

The app-tier EC2 reaches RDS over the private network on port 5432, allowed only because database-sg accepts traffic from appserver-sg — never public. Connection string: `psql -h <rds-endpoint> -p 5432 -U paybridge_admin -d paybridge`. I tested this live from the app-tier instance and it connected successfully over SSL.



```

[ec2-user@ip-10-0-143-79 ~]$ psql -h paybridge-db.croa4k60e1mi.eu-west-1.rds.amazonaws.com -p 5432 -U paybridge_admin -d paybridge
Password for user paybridge_admin:
psql: error: connection to server at "paybridge-db.croa4k60e1mi.eu-west-1.rds.amazonaws.com" (10.0.163.197), port 5432 failed: FATAL:
password authentication failed for user "paybridge_admin"
connection to server at "paybridge-db.croa4k60e1mi.eu-west-1.rds.amazonaws.com" (10.0.163.197), port 5432 failed: FATAL: no pg_hba.c
onf entry for host "10.0.143.79", user "paybridge_admin", database "paybridge", no encryption
[ec2-user@ip-10-0-143-79 ~]$ psql -h paybridge-db.croa4k60e1mi.eu-west-1.rds.amazonaws.com -p 5432 -U paybridge_admin -d paybridge
Password for user paybridge_admin:
psql (15.18)
SSL connection (protocol: TLSv1.2, cipher: ECDHE-RSA-AES256-GCM-SHA384, compression: off)
Type "help" for help.

paybridge=> |
  
```

Figure 5.4: psql connection from app-tier EC2 (10.0.143.79) to paybridge-db, showing SSL connection established and the paybridge=> prompt

Q3: What is the purpose of the DB Subnet Group? What would happen if you only added one subnet (one AZ)?

A DB Subnet Group tells RDS which subnets it's allowed to use, and it needs at least two AZs for Multi-AZ to work. With only one subnet, Multi-AZ would be impossible — there'd be nowhere to put the standby. Mine spans eu-west-1a and eu-west-1b, so it's already Multi-AZ-ready once the account restriction is lifted.

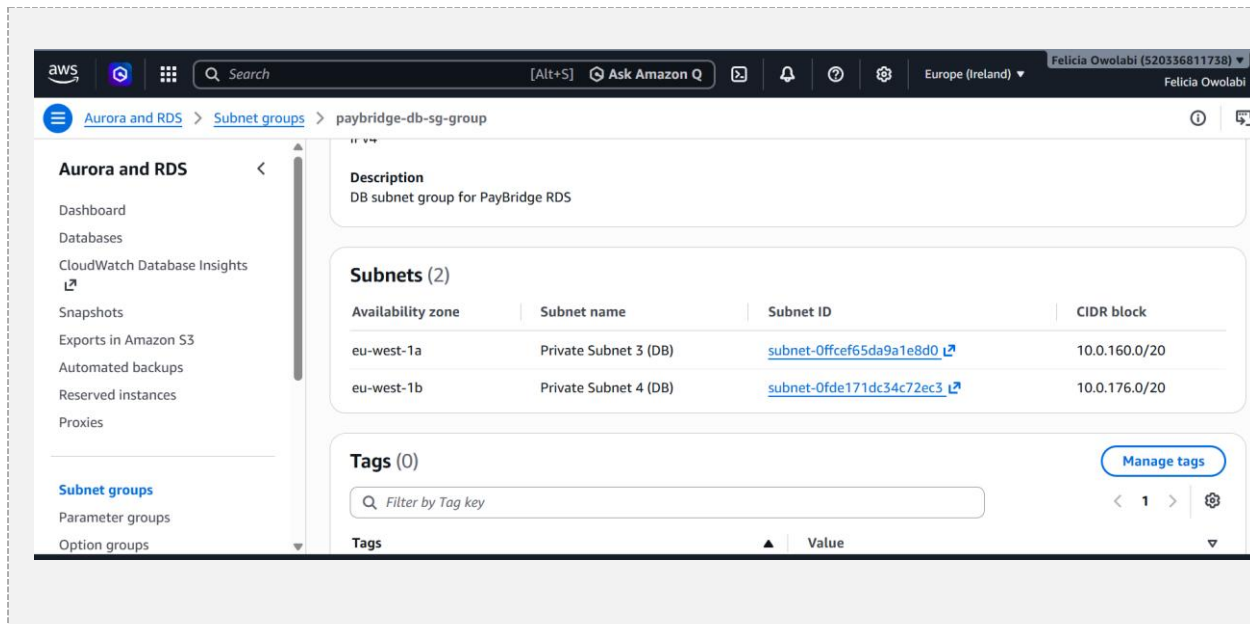


Figure 5.5: paybridge-db-sg-group showing both DB subnets across eu-west-1a and eu-west-1b (2 Availability Zones)

Q4: A developer wants to connect to the RDS instance directly from their laptop using pgAdmin. Outline a secure way to enable this without making the DB public.

I'd never make the database public. Instead I'd SSH tunnel through the bastion, forwarding a local port to the RDS endpoint, so pgAdmin connects to localhost while the bastion relays traffic securely into the VPC. For a whole team, a VPN into the VPC would scale better than tunneling through one bastion.

Section 7 — Task 6: Provisioning Method Justification

This project was provisioned entirely through the AWS Management Console (Option A), a deliberate choice given that the primary goal was to build depth of understanding of each individual AWS service — VPC, subnets, route tables, security groups, launch templates, ASGs, ALBs, and RDS — rather than to abstract that detail away behind Infrastructure as Code from the start. Working directly in the Console meant every setting had to be actively chosen and understood: for example, the account-level Free Plan restriction on RDS Multi-AZ (Task 5) surfaced as a real, visible constraint precisely because the Console exposes every option and every failure message directly, which a CLI script or Terraform apply might otherwise mask behind a generic provisioning error. For a first full build of a 3-tier architecture, this trade-off of speed for understanding was the right one; the CLI and Terraform reflection questions below are answered based on how this same build would have differed under those approaches.

Q1: What are the main risks of provisioning infrastructure manually via the Console compared to using IaC?

The most significant risk is configuration drift with no source of truth: every resource in this project — paybridge-vpc, its six subnets, four security groups, two launch templates, two ASGs, two ALBs, and paybridge-db — exists only as clicks that were made in a browser, with no file anywhere that fully describes the intended end state. If any single setting were changed accidentally later (for example, someone widening database-sg's inbound rule during troubleshooting), there is no diff to review and no automated way to detect or revert the change; it would have to be caught by manual inspection.

This project also experienced this risk directly and repeatedly during the build: the RDS creation form defaulted to the account's Default VPC rather than `paybridge-vpc`, and defaulted the DB subnet group to “default” rather than `paybridge-db-sg-group`, on the very first attempt. In a Console-driven build, catching that requires a human to notice a dropdown value before clicking Create; in Terraform, the correct VPC and subnet group would be hardcoded as variables, and the plan output would make any deviation from the intended values visible before anything is created. Manual provisioning is also not repeatable: rebuilding this exact environment in a second AWS account, or in `us-east-1` instead of `eu-west-1`, would mean manually repeating every Console step from scratch, with every opportunity for the same kind of small mismatch to occur again, whereas an IaC approach would rebuild identically from the same code.

Q2: What does idempotent mean in the context of infrastructure provisioning? Give a concrete example.

Idempotent means that applying the same operation multiple times produces the same end result as applying it once — running it a second, third, or hundredth time causes no additional changes and no errors, because the tool checks the current state before acting rather than blindly re-executing every step.

A concrete example from this project: if `paybridge-db-sg-group` already exists with the correct two DB subnets, an idempotent provisioning script re-run against the same configuration would detect that the subnet group already matches the desired state and simply skip creating it, leaving the existing resource untouched. A non-idempotent equivalent — for instance, a plain AWS CLI `aws rds create-db-subnet-group` command run twice with no existence check — would fail with a “already exists” error on the second run, or worse, in a less careful script, could attempt to create a duplicate and cause naming conflicts or unexpected resource sprawl. Terraform achieves idempotency by comparing its state file against the actual infrastructure on every plan and apply, only changing what has actually drifted from the desired configuration.

Q3: If you used Terraform, explain what happens when you run `terraform apply` twice on the same code.

The first `terraform apply` reads the `.tf` configuration, finds no existing state, and creates every resource from scratch — in this project's case, that would mean `paybridge-vpc`, all six subnets, the IGW, NAT Gateway, both route tables, all four security groups, both launch templates, both ASGs, both ALBs and target groups, and finally `paybridge-db`, recording each resource's real AWS ID into a `terraform.tfstate` file as it goes.

Running `terraform apply` a second time with no changes to the `.tf` files causes Terraform to refresh its state by querying AWS for the current status of every resource, compare that against the configuration, find no differences, and report “No changes. Your infrastructure matches the configuration,” making zero API calls to create, modify, or destroy anything. This is exactly the idempotency described in Q2, and it is also the specific verification method the project brief's Option C explicitly asks for — including a `terraform plan` output showing 0 changes after apply as report evidence that the applied infrastructure genuinely matches the code, rather than having drifted from it.

Q4: How would you securely store RDS credentials in your chosen provisioning method (no hardcoded passwords)?

In this Console-based build, the RDS master password for `paybridge_admin` was set directly during the “Create database” wizard under self-managed credentials, and recorded outside of AWS — in a private, non-shared note kept separately from this report and from any file that would be submitted or committed anywhere. It is never written into the report, the architecture diagram, screenshots, or any script, and screenshots taken during the build (such as Figure 5.2 and 5.4) were checked to confirm the password itself is never visible on screen, since terminal password prompts do not echo input.

The more robust, production-appropriate answer — and the approach that would be used if this were rebuilt using CLI or Terraform — is AWS Secrets Manager, which is in fact exactly what Extension Challenge 4 in this project's brief proposes: store the RDS master password as a JSON secret in Secrets Manager, attach an IAM role to the app-tier EC2 instances with `secretsmanager:GetSecretValue` permission, and have the application retrieve the credential at runtime rather than reading it from an environment variable, a .tf file, or a hardcoded string in code. In Terraform specifically, this is done by referencing an `aws_secretsmanager_secret_version` data source rather than embedding the password as a variable default, so the actual secret value never appears in the .tf files or in terraform plan output, and access to it is governed by IAM policy rather than by who can read the source code.

Section 8 — Cost Estimate

The estimate below was built in the AWS Pricing Calculator ([calculator.aws](#)), configured line-by-line to match the actual resources deployed in this project rather than using calculator defaults. Each service was set to Europe (Ireland), On-Demand pricing, and the exact instance types, counts, and deployment options used in the live build — including Single-AZ for RDS, matching the account-level Multi-AZ restriction discussed in Section 6.

Service	Configuration	Monthly Cost
Amazon EC2 (Web tier)	t3.micro, On-Demand, paybridge-web-asg	\$9.02
Amazon EC2 (App tier)	t3.micro × 2, On-Demand, paybridge-app-asg (Desired: 2)	\$16.64
Amazon VPC — NAT Gateway	1 gateway, ~5 GB/month processed (paybridge-nat)	\$35.28
Elastic Load Balancing — External ALB	Application Load Balancer, internet-facing (paybridge-external-alb)	\$24.24
Elastic Load Balancing — Internal ALB	Application Load Balancer, internal (paybridge-internal-alb)	\$24.24
Amazon RDS for PostgreSQL	db.t3.micro, Single-AZ, 20 GB gp2, On-Demand (paybridge-db)	\$17.14
Total	6 services, Europe (Ireland)	\$126.56 / mo

[INSERT SCREENSHOT HERE]

AWS Pricing Calculator "My Estimate" page, showing all six line items and the total monthly cost

Figure 6.1: AWS Pricing Calculator "My Estimate" summary — six services totaling \$126.56/month (\$1,518.72 over 12 months), all in Europe (Ireland)

The single largest line item is the NAT Gateway at \$35.28/month — more than either Application Load Balancer and nearly 4× the cost of the web-tier EC2 instance — despite carrying almost no real data in this demo. That is a function of AWS's per-hour NAT Gateway charge applying regardless of traffic volume, and it is the clearest illustration in this cost estimate of a resource that is architecturally necessary (private-tier outbound internet access, per Section 2) but disproportionately expensive for a low-traffic development environment relative to what it is actually doing.

Section 9 — Production Considerations

This build was intentionally scoped as a dev-tier demonstration, not a production-ready deployment. Before PayBridge could handle real fintech traffic, I would make the following changes:

- **Enable Multi-AZ on RDS** — Single-AZ was forced by the Free Plan (Section 6); the DB subnet group already spans two AZs, so this is a config change, not a redesign.
- **Add a WAF in front of the external ALB** — for application-layer protection (SQL injection, XSS) that Security Groups can't provide.
- **Enforce HTTPS** — both ALBs are HTTP-only; production needs an ACM cert on the external ALB with an HTTP→HTTPS redirect.
- **Move credentials to Secrets Manager** — the RDS password is currently in a private note (Section 7 Q4); Secrets Manager plus an IAM role removes the human-held copy.
- **Add real Auto Scaling policies** — both ASGs have no scaling policy today (Section 4 Q2); target-tracking on CPU or request count would make them actually elastic.
- **Extend backups and confirm encryption at rest** — retention was capped at 1 day by the Free Plan; production needs 7+ days plus verified RDS/EBS encryption.
- **Move to Infrastructure as Code** — rebuild in Terraform with version control and a CI/CD plan-review step, closing the drift/repeatability gaps from Section 7 Q1.